

|                         |                                  |
|-------------------------|----------------------------------|
| <b>Begonnen am</b>      | Mittwoch, 22. Januar 2025, 23:09 |
| <b>Status</b>           | Beendet                          |
| <b>Beendet am</b>       | Freitag, 24. Januar 2025, 11:37  |
| <b>Verbrauchte Zeit</b> | 1 Tag 12 Stunden                 |

#### Information

## Hinweise zu den Übungen

Mit den Übungen können Sie das erlernte Wissen und Können festigen.

Gegebenenfalls gibt es "Advanced" Fragen, welche zur weiteren Vertiefung für interessierte Studierende dienen. Diese Aufgaben sind nicht Teil des Prüfungsstoffes im engeren Sinn und als "Advanced" gekennzeichnet..

Bei den meisten Fragen können Sie direkt prüfen, ob Ihre Antwort richtig ist. Gewisse Fragen jedoch (z.B. die offenen Fragen) bieten diese Möglichkeit nicht, weil für diese keine automatisierte Prüfung möglich ist.

Nach Abschluss der gesamten Übung bekommen Sie zusätzlich zu Ihren Resultaten bei etlichen Aufgaben allgemeine Hinweise über die Lösung.

**Frage 1**

Nicht

beantwortet

Nicht bewertet

## Write code

Write code to perform the same as a BHI, but using flag dependent branch instructions (no arithmetic branch instruction)

### Explanations

Some processors do not have arithmetic conditional branch operations. You need to combine flags yourself to get the needed results.

BHI requires 2 flags to be tested. C must be 1 and Z must be 0

|    |                 |                   |
|----|-----------------|-------------------|
| HI | Unsigned higher | C == 1 and Z == 0 |
|----|-----------------|-------------------|

There are several ways of implementing the code. Here are 2 of them:

- a) You can read the flags in a register and then perform a test with the appropriate mask for those flags you require set or cleared
- b) You can use several flag dependent tests, checkin everytime the right value for the appropriate flag. But you should be careful not to wrongly change flags with the program operations.

for instance:

BCC else ; jump to else case if C==0

BEQ else ; jump to else case if Z==1

true ...

else ...

**Frage 2**

Nicht

beantwortet

Nicht bewertet

## Understanding the theory and reasons behind the indirect branch

Consider again the indirect branch operation and answer the following questions:

1. What is the difference with the direct unconditional operations as far as the position of the branch target is concerned? (how do these operations allow you to jump)
2. What is the difference at run time between indirect and direct branch operations? (as far as the flexibility of the target is concerned)
- 3: Considering the 2 questions above, when will you use indirect branches?



### Explanations:

1. The indirect branch gives you a wider branch field (direct branch has limited range because offset is stored in the instruction)
2. The target of the direct branch is fixed at compile time. It is known and fixed in the code and does not change in run time unless the code is changed. The target for the indirect branch can change at run time. It is in a register and changing the value in that register changes the target. It is more flexible and also requires more caution in its use.

**Frage 3**

Richtig

Nicht bewertet

## Understanding the theory behind branch operations

How many values could the program counter take following the execution of the given branch operations?

Assume that the flags and R0 values change.

|            |                                          |
|------------|------------------------------------------|
| BVS label3 | <input type="text" value="2"/>           |
| BAL label2 | <input type="text" value="1"/>           |
| B label4   | <input type="text" value="1"/>           |
| BX R0      | <input type="text" value="More than 2"/> |
| BEQ label1 | <input type="text" value="2"/>           |

Your answer is correct.

### Explanations. Check again in your slides.

Conditional branches (Bcc label) can be taken or not taken. It means that depending on the evaluation of the flags, there could be a branch (PC modified accordingly), or no branch (PC keeping its value). Therefore, 2 possible values for the PC.

Unconditional relative operation (B label, BL label or Bal label) always jump to the target. The PC is relatively modified by label after the execution of that operation. Therefore, only 1 possible value for the PC.

Absolute branch operations (BX Rm or BLX Rm) for the Cortex M0 result in the PC being overwritten with the value that is in register Rm at run time ( $PC = Rm$ ). If Rm changes, the target also changes. There are therefore many possibilities for the PC as Rm varies.

Die richtige Antwort ist:

BVS label3 → 2,

BAL label2 → 1,

B label4 → 1,

BX R0 → More than 2,

BEQ label1 → 2

Frage 4

Richtig

Nicht bewertet

## Understanding the Theory Behind Branch Operations

Select the correct answer for the following questions.

Rm is a meaningful register for the operation.

label is a meaningful label for the operation.

Which of the following branch operations allows branching accross the whole memory map with one operation?

BX Rm

Which of the following branch operations has to the shortest maximal branch distance ?

BCC label

Your answer is correct.

### Explanations. Check your documents.

To branch (or jump) accross the whole memory map, it must be possible to change the PC accordingly. That means to cover all portions of the memory map..

For the Cortex M0, this can only be done with the absolute branch operation, where the PC is replaced by a 32-bit word of a register.

That is the BX Rm operation, or the BLX Rm operation

BCC operations allow the PC to be relatively changed. Those changes are in a small region around the current PC (boundaries are PC-256; PC+254)

Die richtige Antwort ist:

Which of the following branch operations allows branching accross the whole memory map with one operation? → BX Rm,

Which of the following branch operations has to the shortest maximal branch distance ? → BCC label

## Frage 5

Nicht  
beantwortet

Nicht bewertet

## Understanding the Theory Behind Branch Operations

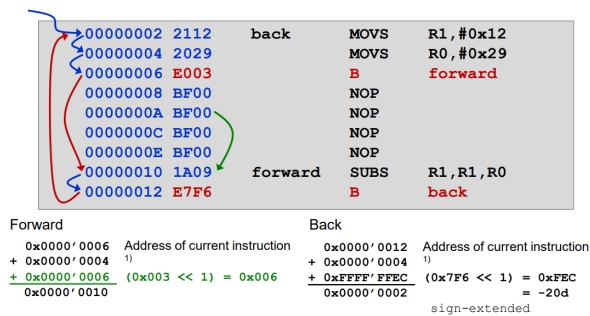
For the following branch operation, calculate the contents of the PC after execution of the operation.

Assume that because of pipeline, PC is always current address plus 4. all number are given in hexadecimal.

Address of instruction: 00000006 Machine code of branch instruction: E003 Contents of PC (in hex, with all 0s and without 0x)

Address of instruction: 00000012 Machine code of branch instruction: E7F6 Contents of PC (in hex, with all 0s and without 0x)

## Explanations. Check your documents.



<sup>1)</sup> Because of pipeline, PC is always current address plus 4

Frage 6

Richtig

Nicht bewertet

## Understanding the Theory Behind Branch Operations

Classify the following branch operation

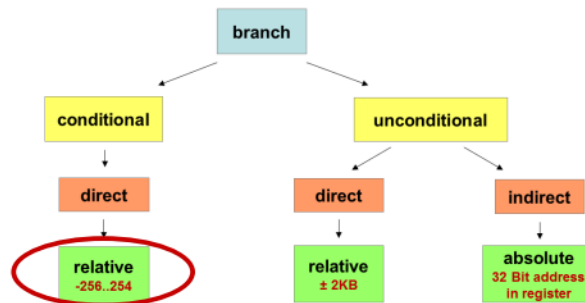
BCS label1

Condition  Addressing Mode

.

### Explanations. Check your documents.

BCS label branches if Carry flag is set (Conditional). The target address is computed using the label which is known at compiling time (direct addressing). The label is added to the PC in case of branching (relative branch)



Frage 7

Richtig

Nicht bewertet

## Understanding the Theory Behind Branch Operations

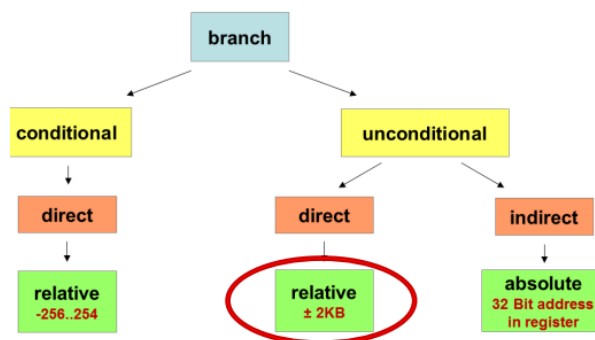
Classify the following branch operation

B label1

Condition  Addressing Mode  
   .

### Explanations. Check your documents.

B label1 always branches (unconditional). It uses a known label position to compute the target address (Direct). The label is added to the PC value (Relative branch)





Frage 8

Richtig

Nicht bewertet

## Understanding the Theory Behind Branch Operations

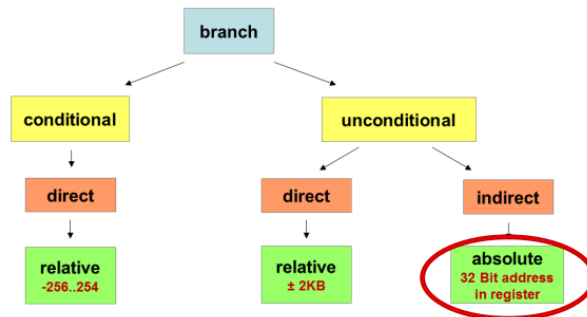
Classify the following branch operation

BX R5:

Condition  Addressing Mode  
   .

### Explanations. Check your documents.

BX R5 always branches (unconditional). It uses data that is in a register to compute the target address (Indirect). The PC is totally replaced by the value of R5 (Absolute branch)



Frage 9

Richtig

Nicht bewertet

## Understanding the theory behind Branch Operations

Which Cortex-M0 register is most likely to be modified by a branch operation?

- ☐ R3
- ☐ R1
- ☐ R13
- ☐ R9
- ☐ R7
- ☐ None
- ☐ R4
- ☐ R12
- ☐ R2
- ☐ R8
- ☒ R15
- ☐ R5
- ☐ R0
- ☐ R11
- ☐ R14
- ☐ R10
- ☐ R6

Your answer is correct.

**Explanations. Check again in your document.**

The Branch instruction when executed modifies the program Counter (PC).

On the Cortex-M0, this is register R15

Die richtige Antwort ist:

R15

## Understanding the Theory Behind Branch Operations

Select all the flags that are important in determining where the next instruction is fetched from, when the following branch instruction is executed.

BLT label1

Wählen Sie eine oder mehrere Antworten:

- ☐ None
- ☒ Overflow (V)
- ☒ Negative (N)
- ☐ Zero (Z)
- ☐ Carry (C)

Your answer is correct.

### Explanations. Check your documents.

See various tables showing flags that are checked. One such table is shown below.

As example, BLT label1 requires the testing of N and V flags.

| cond | short | Flag   |
|------|-------|--------|
| 0000 | EQ    | Z == 1 |
| 0001 | NE    | Z == 0 |
| 0010 | CS/HS | C == 1 |
| 0011 | CC/LO | C == 0 |
| 0100 | MI    | N == 1 |
| 0101 | PL    | N == 0 |
| 0110 | VS    | V == 1 |
| 0111 | VC    | V == 0 |

| cond | short | Flag              |
|------|-------|-------------------|
| 1000 | HI    | C == 1 and Z == 0 |
| 1001 | LS    | C == 0 or Z == 1  |
| 1010 | GE    | N == V            |
| 1011 | LT    | N != V            |
| 1100 | GT    | Z == 0 and N == V |
| 1101 | LE    | Z == 1 or N != V  |
| 1110 | AL    | always            |
| 1111 | --    | --                |

Die richtigen Antworten sind:

Negative (N),

Overflow (V)

Frage 11

Richtig

Nicht bewertet

## Understanding the Theory Behind Branch Operations

Select in the list all the elements that are important in determining where the next instruction is fetched from, when the following branch instruction is executed.

BX R1

Wählen Sie eine oder mehrere Antworten:

- ☒ R1
- ☐ None
- ☐ R13
- ☐ R0
- ☐ R14
- ☐ Flags
- ☐ R15

Your answer is correct.

**Explanations. Check your documents.**

The BX R2 operates a change of the PC into R2 ( $PC = R2$ )

The important element in the list is R2

Die richtige Antwort ist:

R1

**Frage 12**

Richtig

Nicht bewertet

## Understanding the Theory Behind Branch Operations

Select all the flags that are important in determining where the next instruction is fetched from, when the following branch instruction is executed.

BAL label1

Wählen Sie eine oder mehrere Antworten:

- ☒ None
- ☐ Zero (Z)
- ☐ Carry (C)
- ☐ Negative (N)
- ☐ Overflow (V)

Your answer is correct.

### Explanations. Check your documents.

See various tables showing flags that are checked. One such table is shown below.

As example, BAL label1 does not depend on any flag. It always branches.

| cond | short | Flag   |
|------|-------|--------|
| 0000 | EQ    | Z == 1 |
| 0001 | NE    | Z == 0 |
| 0010 | CS/HS | C == 1 |
| 0011 | CC/LO | C == 0 |
| 0100 | MI    | N == 1 |
| 0101 | PL    | N == 0 |
| 0110 | VS    | V == 1 |
| 0111 | VC    | V == 0 |

| cond | short | Flag              |
|------|-------|-------------------|
| 1000 | HI    | C == 1 and Z == 0 |
| 1001 | LS    | C == 0 or Z == 1  |
| 1010 | GE    | N == V            |
| 1011 | LT    | N != V            |
| 1100 | GT    | Z == 0 and N == V |
| 1101 | LE    | Z == 1 or N != V  |
| 1110 | AL    | always            |
| 1111 | --    | --                |

Die richtige Antwort ist:

None

◀ Übungen Logic  
and Shift/Rotate  
Instructions

Direkt zu:

Lecture Control  
Structures ▶